

ARRAY QNO:3

QUESTION:

Problem Description:

Rigesh is an electronic shop owner. Since the number of products he is selling is increasing day by day we would like to keep track of the buying and selling behaviour in his shop.

So given the cost of stock on each day in an array $A[]$ of size N . Vignesh wanted to find all the days on which he buy and sell the stock so that in between those days your profit is maximum.

Constraints:

$1 \leq t \leq 10$

$1 \leq n \leq 10$

Input Format:

First line contains number of test cases T .

First line of each test case contains an integer value N denoting the number of days, followed by an array of stock prices of N days.

Output Format:

For each testcase, output all the days with profit in a single line.

If there is no profit then print "No Profit".

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
1
5
130 204 150 175 140
```

EXPECTED OUTPUT

```
(0 1) (2 3)
```

Test Case 2

INPUT (STDIN)

```
2
5
131 124 110 175 110
3
23 45 67
```

EXPECTED OUTPUT

```
(2 3)
(0 2)
```

▼ Mandatory Test Cases

Test Case 1
KEYWORD
<code>if(arr[i]>arr[i-1])</code>

▼ Complexity Test Cases

Test Case 1
CYCLOMATIC COMPLEXITY
17

Test Case 2
TOKEN COUNT
350

Test Case 3
NLOC
71

Source Code:

```
#include <stdio.h>

void calculateProfit() {
    int n;
    scanf("%d", &n);
    int arr[15];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int buy = -1;
    int found = 0;

    for (int i = 1; i < n; i++) {
        if(arr[i]>arr[i-1]) {
            if (buy == -1) {
                buy = i - 1;
            }
        }

        if (buy != -1 && (i == n - 1 || arr[i+1] < arr[i])) {
            if (found == 1) {
                printf(" ");
            }

            printf("(%d%d)", buy, i);
            found = 1;
        }
    }
}
```

```

        buy = -1;
    }
}

if (!found) {
    printf("No Profit");
}
printf("\n");
}

int main() {
    int t;
    if (scanf("%d", &t) == 1) {
        while (t > 0) {
            calculateProfit();
            t--;
        }
    }
    return 0;
}

```

Output:

```

1
5
130 204 150 175 140
(01) (23)

```

```

2
5
131 124 110 175 110
(23)
3
24 45 67
(02)

```